```
package com.example.aurawall

import android.content.Context
import android.net.Uri
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.compose.setContent
import androidx.activity.result.PickVisualMediaRequest
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.pager.HorizontalPager
import androidx.compose.foundation.pager.rememberPagerState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.PlayArrow
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.util.lerp
import androidx.work.PeriodicWorkRequestBuilder
import androidx.work.WorkManager
import coil.compose.AsyncImage
import java.text.SimpleDateFormat
import java.util.*
import java.util.concurrent.TimeUnit
import kotlin.math.absoluteValue
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme(colorScheme = darkColorScheme(background = Color(0xFF0C0C12))) {
                WallpaperStudioScreen()
            }
        }
    }
}

@Composable
fun WallpaperStudioScreen() {
    val context = LocalContext.current
    val prefs = remember { context.getSharedPreferences("aura_prefs",
Context.MODE_PRIVATE) }

    val wallpaperList = remember {
        mutableStateListOf(
            "https://images.unsplash.com/photo-1618005182384-a83a8bd57fbe?w=800",
            "https://images.unsplash.com/photo-1506744038136-46273834b3fb?w=800",
            "https://images.unsplash.com/photo-1518709268805-4e9042af9f23?w=800"
        )
    }

    val pickerLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.PickMultipleVisualMedia()
    ) { uris: List<Uri> ->
        uris.forEach { uri -> wallpaperList.add(uri.toString()) }
        prefs.edit().putStringSet("wallpaper_pool", wallpaperList.toSet()).apply()
    }

    val pagerState = rememberPagerState(pageCount = { wallpaperList.size })
    var selectedDurationHours by remember { mutableStateOf(1L) }
    var isSchedulerActive by remember { mutableStateOf(false) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(Color(0xFF0C0C12))
            .padding(top = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Row(
```

```
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 24.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text("AuraWall", fontSize = 28.sp, fontWeight = FontWeight.Bold, color = Color.White)
                Text("Auto-Rotation Studio", fontSize = 14.sp, color = Color(0xFF8E8E9B))
            }

            IconButton(
                onClick = {
pickerLauncher.launch(PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.Imag
eOnly)) },
                modifier = Modifier.clip(CircleShape).background(Color(0xFF232330))
            ) {
                Icon(Icons.Default.Add, contentDescription = "Add Photos", tint = Color(0xFF6366F1))
            }
        }

        Spacer(modifier = Modifier.height(24.dp))

        HorizontalPager(
            state = pagerState,
            contentPadding = PaddingValues(horizontal = 56.dp),
            modifier = Modifier.weight(1f)
        ) { page ->
            val pageOffset = ((pagerState.currentPage - page) +
pagerState.currentPageOffsetFraction).absoluteValue

            Card(
                shape = RoundedCornerShape(28.dp),
                modifier = Modifier
                    .fillMaxSize()
                    .graphicsLayer {
                        val scale = lerp(0.85f, 1f, 1f - pageOffset.coerceIn(0f, 1f))
                        scaleX = scale
                        scaleY = scale
                        alpha = lerp(0.5f, 1f, 1f - pageOffset.coerceIn(0f, 1f))
                        rotationY = (pageOffset * 15f) * if (page < pagerState.currentPage) 1f else -1f
                    }
                    .border(2.dp, Color(0xFF282838), RoundedCornerShape(28.dp))
```

```
                .shadow(elevation = 20.dp, ambientColor = Color(0xFF6366F1), spotColor =
Color(0xFF6366F1))
            ) {
                Box(modifier = Modifier.fillMaxSize()) {
                    AsyncImage(
                        model = wallpaperList[page],
                        contentDescription = null,
                        contentScale = ContentScale.Crop,
                        modifier = Modifier.fillMaxSize()
                    )

                    Column(
                        modifier = Modifier.fillMaxWidth().padding(top = 40.dp),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        val currentTime = SimpleDateFormat("HH:mm",
Locale.getDefault()).format(Date())
                        val currentDate = SimpleDateFormat("EEE, d MMM",
Locale.getDefault()).format(Date())

                        Text(currentTime, fontSize = 54.sp, fontWeight = FontWeight.Light, color =
Color.White)
                        Text(currentDate, fontSize = 16.sp, color = Color.White.copy(alpha = 0.8f))
                    }
                }
            }
        }

        Spacer(modifier = Modifier.height(20.dp))

        Row(
            modifier = Modifier.fillMaxWidth().padding(horizontal = 24.dp),
            horizontalArrangement = Arrangement.SpaceEvenly
        ) {
            listOf(1L to "1h", 3L to "3h", 12L to "12h", 24L to "24h").forEach { (hours, label) ->
                val isSelected = selectedDurationHours == hours
                Box(
                    modifier = Modifier
                        .clip(RoundedCornerShape(14.dp))
                        .background(if (isSelected) Color(0xFF6366F1) else Color(0xFF1E1E28))
                        .clickable { selectedDurationHours = hours }
                        .padding(horizontal = 20.dp, vertical = 10.dp)
                ) {
```

```
                Text(label, color = if (isSelected) Color.White else Color(0xFF9E9EB0), fontWeight
= FontWeight.Bold)
            }
        }
    }

    Spacer(modifier = Modifier.height(16.dp))

    Button(
        onClick = {
            prefs.edit().putStringSet("wallpaper_pool", wallpaperList.toSet()).apply()

            if (!isSchedulerActive) {
                val workRequest = PeriodicWorkRequestBuilder<WallpaperWorker>(
                    selectedDurationHours, TimeUnit.HOURS
                ).build()
                WorkManager.getInstance(context).enqueue(workRequest)
                isSchedulerActive = true
            } else {
                WorkManager.getInstance(context).cancelAllWork()
                isSchedulerActive = false
            }
        },
        colors = ButtonDefaults.buttonColors(
            containerColor = if (isSchedulerActive) Color(0xFF10B981) else Color(0xFF6366F1)
        ),
        shape = RoundedCornerShape(20.dp),
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 24.dp, vertical = 12.dp)
            .height(56.dp)
    ) {
        Icon(if (isSchedulerActive) Icons.Default.Check else Icons.Default.PlayArrow,
contentDescription = null)
        Spacer(modifier = Modifier.width(8.dp))
        Text(if (isSchedulerActive) "Active (Every $selectedDurationHours hr)" else "Activate
Auto-Rotation")
    }
  }
}
```